Machine Learning & Deep Learning Lab

16 Programs with Line-by-Line Explanations and Running Procedures

Note: All programs require Python 3.7+ with libraries installed.

Install required libraries

```
pip install numpy pandas matplotlib scikit-learn tensorflow keras seaborn
```

Program 1: Data Preprocessing — Pima Indians Diabetes

Perform preprocessing on healthcare dataset: handle missing values, encode categorical variables, normalize, and analyze with summary statistics and visualizations.

Complete Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Download dataset: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database
df = pd.read_csv('diabetes.csv')

# 1. Handle missing values (zeros in certain columns represent missing)
cols_with_zeros = ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI']
for col in cols_with_zeros:
    df[col] = df[col].replace(0, np.nan)  # Replace 0 with NaN
    df[col] = df[col].fillna(df[col].median())  # Fill with median

# 2. Summary statistics
print("Summary Statistics:")
print(df.describe())
print(f"\nMissing values:\n{df.isnull().sum()}")

# 3. Normalization (Min-Max scaling)
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
feature_cols = ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness',
                'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age']
df[feature_cols] = scaler.fit_transform(df[feature_cols])
print("\nAfter normalization:\n", df.describe())

# 4. Visualizations
plt.figure(figsize=(15, 10))
plt.subplot(221); df['Outcome'].value_counts().plot(kind='bar'); plt.title('Class Distribution')
plt.subplot(222); sns.histplot(df['Glucose'], bins=30); plt.title('Glucose Distribution')
plt.subplot(223); sns.boxplot(x='Outcome', y='Age', data=df); plt.title('Age vs Outcome')
plt.subplot(224); corr = df.corr(); sns.heatmap(corr, annot=True, cmap='coolwarm'); plt.title('Correlation Matrix')
plt.tight_layout(); plt.show()
```

Line-by-Line Explanation

`pd.read_csv("diabetes.csv")`	**Load dataset into pandas DataFrame**
`df[col].replace(0, np.nan)`	Replace zeros with NaN (0 is invalid for Glucose, BMI, etc.)
`df[col].fillna(df[col].median())`	Fill missing values with median (robust to outliers)
`df.describe()`	Summary: count, mean, std, min, 25%, 50%,

	75%, max for each column
`MinMaxScaler()`	Scale all features to [0,1] range: (x - min) / (max - min)
`scaler.fit_transform(df[feature_cols])`	Compute min/max from data then transform
`sns.heatmap(df.corr(), annot=True)`	Correlation matrix with numerical annotations
`sns.boxplot(x="Outcome", y="Age", data=df)`	Box plot comparing Age distribution for diabetic vs non-diabetic

Running Procedure

1. Download diabetes.csv from Kaggle and place in D:\allen\

2. Run: python program1_preprocessing.py

3. View console output for statistics and matplotlib window for plots

Program 2: Linear Regression — Auto MPG

Predict fuel efficiency (MPG) of automobiles and evaluate using MSE and R-squared.

Complete Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Load dataset
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/auto-mpg/auto-mpg.data'
columns = ['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
           'acceleration', 'model_year', 'origin', 'car_name']
df = pd.read_csv(url, names=columns, delim_whitespace=True, na_values='?')

# Handle missing values
df = df.dropna()
df = df.drop('car_name', axis=1)  # Drop non-numeric column

# Split features and target
X = df.drop('mpg', axis=1)
y = df['mpg']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

# Evaluate
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error: {mse:.4f}")
print(f"R-squared Score: {r2:.4f}")
print(f"Coefficients: {dict(zip(X.columns, model.coef_))}")
print(f"Intercept: {model.intercept_:.4f}")

# Plot
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, alpha=0.6)
plt.plot([y.min(), y.max()], [y.min(), y.max()], 'r--', lw=2)
plt.xlabel('Actual MPG'); plt.ylabel('Predicted MPG')
plt.title('Linear Regression: Actual vs Predicted MPG')
plt.show()
```

Line-by-Line Explanation

`pd.read_csv(url, names=columns, delim_whitespace=True)`	**Read space-separated file with column names**
`na_values="?"`	Treat ? as missing values
`df.dropna()`	Remove rows with any missing values
`train_test_split(X, y, test_size=0.2)`	80% training, 20% testing split
`LinearRegression().fit(X_train, y_train)`	OLS: minimize sum of squared residuals
`mean_squared_error(y_test, y_pred)`	MSE = mean((actual - predicted))
`r2_score(y_test, y_pred)`	R-squared = 1 - (residual_variance / total_variance)
`plt.scatter(y_test, y_pred)`	Scatter plot: points on diagonal = perfect predictions

Running Procedure

1. Run: python program2_linear_regression_mpg.py

2. Observe MSE, R^2, coefficients, and intercept

3. View Actual vs Predicted scatter plot

Program 3: Logistic Regression — Breast Cancer

Predict malignant/benign tumor using Logistic Regression with confusion matrix evaluation.

Complete Program

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)
y = cancer.target

print(f"Dataset: {X.shape[0]} samples, {X.shape[1]} features")
print(f"Classes: {cancer.target_names}")

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Train Logistic Regression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

# Evaluate
acc = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Confusion Matrix:\n{cm}")
print(f"\nClassification Report:\n{classification_report(y_test, y_pred,
target_names=cancer.target_names)}")

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=cancer.target_names, yticklabels=cancer.target_names)
plt.xlabel('Predicted'); plt.ylabel('True'); plt.title('Confusion Matrix')
plt.show()
```

Line-by-Line Explanation

`load_breast_cancer()`	**Load Wisconsin breast cancer dataset (569 samples, 30 features)**
`LogisticRegression(max_iter=1000)`	Logistic Regression with increased

	iterations for convergence
`model.fit(X_train, y_train)`	Learn decision boundary using maximum likelihood estimation
`predict_proba(X_test)[:, 1]`	Probability of class 1 (malignant) for each sample
`confusion_matrix(y_test, y_pred)`	Table: [TN, FP; FN, TP] showing prediction errors
`classification_report(...)`	Precision, recall, f1-score, support per class
`sns.heatmap(cm, annot=True, fmt="d")`	Heatmap of confusion matrix with counts

Running Procedure

1. Run: python program3_logistic_regression.py

2. Observe accuracy, confusion matrix, and classification report

3. View confusion matrix heatmap

Program 4: KNN Classifier — Glass Identification

Classify glass types based on chemical properties and analyze effect of varying K.

Complete Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt

# Load Glass dataset
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/glass/glass.data'
columns = ['id', 'RI', 'Na', 'Mg', 'Al', 'Si', 'K', 'Ca', 'Ba', 'Fe', 'type']
df = pd.read_csv(url, names=columns)
df = df.drop('id', axis=1)

X = df.drop('type', axis=1)
y = df['type']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Experiment with different K values
k_values = range(1, 21)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    acc = accuracy_score(y_test, knn.predict(X_test))
    accuracies.append(acc)
    print(f"K={k:2d}  Accuracy={acc:.4f}")

best_k = k_values[np.argmax(accuracies)]
print(f"\nBest K: {best_k} with accuracy {max(accuracies):.4f}")

# Final model
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train, y_train)
y_pred = final_knn.predict(X_test)
print(f"\nClassification Report:\n{classification_report(y_test, y_pred)}")

# Plot
plt.figure(figsize=(10, 6))
plt.plot(k_values, accuracies, 'bo-', linewidth=2, markersize=8)
plt.axvline(x=best_k, color='r', linestyle='--', alpha=0.5)
plt.xlabel('K Value'); plt.ylabel('Accuracy')
plt.title('Glass Classification: Accuracy vs K')
plt.grid(True); plt.show()
```

Line-by-Line Explanation

`pd.read_csv(url, names=columns)`	**Load UCI Glass dataset with column names**
`df.drop("id", axis=1)`	Remove ID column (not a feature)
`KNeighborsClassifier(n_neighbors=k)`	KNN: classifies based on majority vote of K nearest neighbors
`for k in k_values`	Test K=1 through 20 to find optimal value
`np.argmax(accuracies)`	Find K index that gives highest accuracy
`plt.axvline(x=best_k, color="r", ls="--")`	Red dashed line marking best K on plot

Running Procedure

1. Run: python program4_knn_glass.py

2. Observe accuracy for each K value and identify best K

3. View accuracy vs K plot

Program 5: Decision Tree — Bank Marketing

Predict if bank customer will subscribe to a term deposit using Decision Tree.

Complete Program

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, classification_report
import matplotlib.pyplot as plt

# Load dataset
url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/00222/bank-
additional.zip'
# Note: Download and extract bank-additional-full.csv manually
df = pd.read_csv('bank-additional-full.csv', sep=';')

# Convert target: 'yes'/'no' -> 1/0
df['y'] = df['y'].map({'yes': 1, 'no': 0})

# Select numeric columns only for simplicity
numeric_cols = df.select_dtypes(include=[np.number]).columns.tolist()
X = df[numeric_cols[:-1]]  # All numeric except target
y = df['y']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)

# Decision Tree
dt = DecisionTreeClassifier(max_depth=5, criterion='entropy', random_state=42)
dt.fit(X_train, y_train)
y_pred = dt.predict(X_test)

acc = accuracy_score(y_test, y_pred)
prec = precision_score(y_test, y_pred)

print(f"Accuracy: {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"\nClassification Report:\n{classification_report(y_test, y_pred)}")

# Feature importance
feat_imp = sorted(zip(X.columns, dt.feature_importances_), key=lambda x: x[1],
reverse=True)
print("\nTop 5 Features:")
for name, imp in feat_imp[:5]:
    print(f"  {name}: {imp:.4f}")

# Tree visualization
plt.figure(figsize=(20, 12))
plot_tree(dt, filled=True, feature_names=X.columns.tolist(), rounded=True,
max_depth=3)
```

```
plt.title('Decision Tree (Bank Marketing)')
plt.show()
```

Line-by-Line Explanation

`df["y"].map({"yes":1, "no":0})`	**Convert categorical target to binary numeric**
`select_dtypes(include=[np.number])`	Select only numeric columns for simplicity
`DecisionTreeClassifier(max_depth=5, criterion="entropy")`	ID3-style tree using information gain, depth limited to 5
`precision_score(y_test, y_pred)`	Precision = TP / (TP+FP): how many predicted positives are correct
`dt.feature_importances_`	Gini importance: how much each feature contributes to splits
`plot_tree(dt, filled=True, max_depth=3)`	Visualize first 3 levels of decision tree

Running Procedure

1. Download bank-additional-full.csv from UCI (or Kaggle)

2. Run: python program5_decision_tree_bank.py

3. Observe accuracy, precision, feature importance, and tree plot

Program 6: Basic Perceptron — AND / OR Functions

Build a basic perceptron for binary classification of AND and OR logic gates.

Complete Program

```
import numpy as np

class Perceptron:
    def __init__(self, learning_rate=0.01, epochs=100):
        self.lr = learning_rate
        self.epochs = epochs
        self.weights = None
        self.bias = None

    def activation(self, x):
        return 1 if x >= 0 else 0  # Step function

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0

        for epoch in range(self.epochs):
            errors = 0
            for idx, x_i in enumerate(X):
                linear_output = np.dot(x_i, self.weights) + self.bias
                y_pred = self.activation(linear_output)
                update = self.lr * (y[idx] - y_pred)
                self.weights += update * x_i
                self.bias += update
                errors += int(update != 0)
            if errors == 0:
                print(f"Converged at epoch {epoch+1}")
                break

    def predict(self, X):
        return np.array([self.activation(np.dot(x, self.weights) + self.bias) for
x in X])

# AND gate
X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

p_and = Perceptron(lr=0.1, epochs=50)
p_and.fit(X_and, y_and)
print("\nAND Gate:")
print(f"Weights: {p_and.weights}, Bias: {p_and.bias}")
print(f"Predictions: {p_and.predict(X_and)}")

# OR gate
X_or = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_or = np.array([0, 1, 1, 1])
```

```
p_or = Perceptron(lr=0.1, epochs=50)
p_or.fit(X_or, y_or)
print("\nOR Gate:")
print(f"Weights: {p_or.weights}, Bias: {p_or.bias}")
print(f"Predictions: {p_or.predict(X_or)}")
```

Line-by-Line Explanation

`class Perceptron`	**Define Perceptron class with fit and predict methods**
`self.activation(x)`	Step function: returns 1 if x>=0, else 0
`self.weights = np.zeros(n_features)`	Initialize weights to zero
`linear_output = np.dot(x_i, weights) + bias`	Compute weighted sum + bias: w1*x1 + w2*x2 + ... + b
`update = lr * (y_true - y_pred)`	Error signal: difference between actual and predicted
`weights += update * x_i`	Update rule: w = w + lr * (y - y_pred) * x
`bias += update`	Bias updated same way (input is implicitly 1)
`errors == 0 → break`	Stop early if all predictions are correct (converged)

Running Procedure

1. Run: python program6_perceptron_gates.py

2. Observe convergence, weights, bias, and predictions for AND and OR

Program 7: Activation Functions — Step, Sigmoid, Tanh, ReLU

Implement and visualize different activation functions.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt

def step(x):
    return np.where(x >= 0, 1, 0)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def tanh(x):
    return np.tanh(x)

def relu(x):
    return np.maximum(0, x)

# Generate input values
x = np.linspace(-10, 10, 1000)

# Compute outputs
y_step = step(x)
y_sigmoid = sigmoid(x)
y_tanh = tanh(x)
y_relu = relu(x)

# Plot
plt.figure(figsize=(14, 10))
functions = [('Step', y_step), ('Sigmoid', y_sigmoid), ('Tanh', y_tanh), ('ReLU',
y_relu)]

for i, (name, y_vals) in enumerate(functions, 1):
    plt.subplot(2, 2, i)
    plt.plot(x, y_vals, 'b-', linewidth=2)
    plt.axhline(y=0, color='gray', linestyle='--', alpha=0.5)
    plt.axvline(x=0, color='gray', linestyle='--', alpha=0.5)
    plt.title(f'{name} Activation')
    plt.xlabel('Input'); plt.ylabel('Output')
    plt.grid(True, alpha=0.3)
    plt.ylim(-1.5, 1.5) if name != 'ReLU' else plt.ylim(-1, 5)

    # Print properties
    print(f"\n{name}:")
    print(f"  Output range: [{y_vals.min():.2f}, {y_vals.max():.2f}]")
    print(f"  Differentiable: {'No' if name == 'Step' else 'Yes'}")
    if name != 'Step':
        print(f"  Derivative: {'σ(x)*(1-σ(x))' if name == 'Sigmoid' else '1-
tanh²(x)' if name == 'Tanh' else '0 if x<0, 1 if x>0'}")

plt.tight_layout(); plt.show()
```

Line-by-Line Explanation

`np.linspace(-10, 10, 1000)`	**Generate 1000 evenly spaced points from -10 to 10**
`step(x): np.where(x>=0, 1, 0)`	Step: output 0 or 1; non-differentiable at 0
`sigmoid(x): 1/(1+exp(-x))`	Sigmoid: S-shaped, outputs in (0,1); gradient saturation issue
`tanh(x): np.tanh(x)`	Tanh: zero-centered, outputs in (-1,1); stronger gradients than sigmoid
`relu(x): np.maximum(0, x)`	ReLU: outputs 0 for negative, x for positive; solves vanishing gradient
`plt.axhline/axvline`	Reference lines at y=0 and x=0

Running Procedure

1. Run: python program7_activation_functions.py

2. View 4 plots comparing activation functions

Program 8: Single-Layer Perceptron — XOR Problem

Demonstrate that single-layer perceptron cannot solve XOR (linear inseparability).

Complete Program

```
import numpy as np

class Perceptron:
    def __init__(self, lr=0.1, epochs=100):
        self.lr = lr; self.epochs = epochs
        self.weights = None; self.bias = None

    def activation(self, x):
        return 1 if x >= 0 else 0

    def fit(self, X, y):
        n_samples, n_features = X.shape
        self.weights = np.zeros(n_features)
        self.bias = 0
        for epoch in range(self.epochs):
            errors = 0
            for idx, x_i in enumerate(X):
                linear = np.dot(x_i, self.weights) + self.bias
                y_pred = self.activation(linear)
                update = self.lr * (y[idx] - y_pred)
                self.weights += update * x_i
                self.bias += update
                errors += int(update != 0)
            if errors == 0:
                print(f"Converged at epoch {epoch+1}")
                return True
        return False  # Did not converge

# XOR dataset (linearly inseparable)
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([0, 1, 1, 0])  # XOR: same->0, different->1

print("XOR Problem - Single Layer Perceptron")
print("=" * 40)
print(f"Input:\n{X_xor}")
print(f"Expected: {y_xor}")

p = Perceptron(lr=0.1, epochs=200)
converged = p.fit(X_xor, y_xor)
predictions = p.predict(X_xor)

print(f"\nWeights: {p.weights}, Bias: {p.bias}")
print(f"Predictions: {predictions}")
print(f"Correct: {np.array_equal(predictions, y_xor)}")
print(f"Converged: {converged}")
print("\nSingle-layer perceptron CANNOT solve XOR because it's linearly
inseparable.")
```

Line-by-Line Explanation

`X_xor = [[0,0],[0,1],[1,0],[1,1]]`	**XOR inputs: 4 combinations of binary inputs**
`y_xor = [0, 1, 1, 0]`	XOR output: 1 when inputs differ, 0 when same
`Perceptron(lr=0.1, epochs=200)`	Single layer perceptron with no hidden layers
`converged = p.fit(X_xor, y_xor)`	Returns False – perceptron cannot separate XOR
`update = lr * (y_true - y_pred)`	Weight update rule identical to Program 6
`np.array_equal(predictions, y_xor)`	Will be False: at least 1 of 4 predictions is wrong

Running Procedure

1. Run: python program8_xor_perceptron.py

2. Observe that perceptron does NOT converge for XOR

3. Understand why XOR needs hidden layers (MLP)

Program 9: Decision Boundary Visualization — 2D Data

Visualize decision boundary of a single-layer perceptron on 2D linearly separable data.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import Perceptron

# Generate 2D linearly separable data
X, y = make_classification(n_samples=100, n_features=2, n_informative=2,
                           n_redundant=0, n_clusters_per_class=1, random_state=42)

# Train perceptron
p = Perceptron(max_iter=100, random_state=42)
p.fit(X, y)

# Create mesh grid for decision boundary
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))
Z = p.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot
plt.figure(figsize=(10, 8))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, edgecolors='k', s=60)

# Plot decision boundary line
w = p.coef_[0]; b = p.intercept_[0]
x_line = np.array([x_min, x_max])
y_line = -(w[0] * x_line + b) / w[1]
plt.plot(x_line, y_line, 'k--', linewidth=2, label='Decision Boundary')

# Equation
equation = f"y = {(-w[0]/w[1]):.2f}x + {(-b/w[1]):.2f}"
print(f"Weights: {w}, Bias: {b}")
print(f"Decision boundary: {w[0]:.2f}x1 + {w[1]:.2f}x2 + {b:.2f} = 0")
print(f"Line equation: {equation}")

plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.title('Perceptron Decision Boundary'); plt.legend()
plt.show()
```

Line-by-Line Explanation

`make_classification(n_samples=100, n_features=2)`	**Generate 100 samples with 2 features (2D for visualization)**
`Perceptron(max_iter=100)`	Scikit-learn's Perceptron implementation
`np.meshgrid(x_range, y_range)`	Create grid of points covering the feature space

`p.predict(np.c_[xx.ravel(), yy.ravel()])`	Classify every point on the grid
`plt.contourf(xx, yy, Z, alpha=0.3)`	Shaded regions show classification areas
`w[0]*x1 + w[1]*x2 + b = 0`	Decision boundary: line equation from weights
`y = -(w[0]*x + b) / w[1]`	Rearrange to y = mx + c form for plotting

Running Procedure

1. Run: python program9_decision_boundary.py

2. Observe the linear decision boundary separating two classes

3. View console output with line equation

Program 10: MLP with One Hidden Layer

Build a Multilayer Perceptron with one hidden layer for classification.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Generate non-linear dataset
X, y = make_moons(n_samples=500, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

# Build MLP with one hidden layer
model = Sequential([
    Dense(16, activation='relu', input_shape=(2,)),   # Hidden layer
    Dense(1, activation='sigmoid')                     # Output layer
])

model.compile(optimizer=Adam(learning_rate=0.01),
              loss='binary_crossentropy',
              metrics=['accuracy'])
model.summary()

# Train
history = model.fit(X_train, y_train, epochs=50, batch_size=32,
                    validation_data=(X_test, y_test), verbose=1)

# Evaluate
train_acc = model.evaluate(X_train, y_train, verbose=0)[1]
test_acc = model.evaluate(X_test, y_test, verbose=0)[1]
print(f"\nTraining Accuracy: {train_acc:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

# Plot decision boundary
x_min, x_max = X[:, 0].min()-0.5, X[:, 0].max()+0.5
y_min, y_max = X[:, 1].min()-0.5, X[:, 1].max()+0.5
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
                     np.arange(y_min, y_max, 0.02))
Z = (model.predict(np.c_[xx.ravel(), yy.ravel()]) >
0.5).astype(int).reshape(xx.shape)

plt.figure(figsize=(12, 5))
plt.subplot(121)
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm, edgecolors='k')
plt.title('MLP Decision Boundary')
```

```
plt.subplot(122)
plt.plot(history.history['accuracy'], label='Train')
plt.plot(history.history['val_accuracy'], label='Test')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.grid(True)
plt.title('Training History')
plt.tight_layout(); plt.show()
```

Line-by-Line Explanation

`make_moons(n_samples=500, noise=0.2)`	**Generate 2D non-linear dataset (two interleaving half circles)**
`Dense(16, activation="relu", input_shape=(2,))`	Hidden layer with 16 neurons, ReLU activation
`Dense(1, activation="sigmoid")`	Output layer: 1 neuron for binary classification
`Adam(learning_rate=0.01)`	Adam optimizer: adaptive learning rate with momentum
`loss="binary_crossentropy"`	Binary cross-entropy: -y*log(p) - (1-y)*log(1-p)
`model.fit(..., validation_data=(X_test, y_test))`	Train with validation set to monitor overfitting
`plt.contourf(xx, yy, Z, alpha=0.3)`	Non-linear decision boundary (unlike Program 9)

Running Procedure

1. Run: python program10_mlp_hidden.py

2. Observe model summary and training progress

3. View non-linear decision boundary and accuracy curves

Program 11: Backpropagation — MLP on Iris

Implement backpropagation to train MLP on Iris dataset.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
from tensorflow.keras.utils import plot_model

# Load Iris
iris = load_iris()
X, y = iris.data, iris.target

# One-hot encode labels
encoder = OneHotEncoder(sparse_output=False)
y_encoded = encoder.fit_transform(y.reshape(-1, 1))

# Split and scale
X_train, X_test, y_train, y_test = train_test_split(X, y_encoded, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Build MLP
model = Sequential([
    Dense(10, activation='relu', input_shape=(4,)),    # Hidden layer
    Dense(10, activation='relu'),                       # Hidden layer
    Dense(3, activation='softmax')                      # Output: 3 classes
])

model.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
              loss='categorical_crossentropy',
              metrics=['accuracy'])
model.summary()

# Train (backpropagation happens here)
history = model.fit(X_train, y_train, epochs=100, batch_size=8,
                    validation_data=(X_test, y_test), verbose=0)

# Evaluate
train_acc = model.evaluate(X_train, y_train, verbose=0)[1]
test_acc = model.evaluate(X_test, y_test, verbose=0)[1]
print(f"Training Accuracy: {train_acc:.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

# Plot loss and accuracy
```

```
plt.figure(figsize=(12, 4))
plt.subplot(121)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Test Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.grid(True)
plt.title('Loss (Backpropagation)')

plt.subplot(122)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Test Acc')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.grid(True)
plt.title('Accuracy')
plt.tight_layout(); plt.show()

print("\nBackpropagation steps per epoch:")
print("1. Forward pass: compute predictions")
print("2. Compute loss: categorical cross-entropy")
print("3. Backward pass: compute gradients via chain rule")
print("4. Update weights: w = w - lr * gradient")
```

Line-by-Line Explanation

`OneHotEncoder()`	**Convert class labels 0,1,2 to [1,0,0], [0,1,0], [0,0,1]**
`StandardScaler()`	Standardize: zero mean, unit variance
`Dense(10, activation="relu")`	Hidden layer with 10 neurons
`Dense(3, activation="softmax")`	Output: 3 neurons, softmax gives probability distribution
`SGD(learning_rate=0.01, momentum=0.9)`	SGD with momentum: accelerates convergence
`loss="categorical_crossentropy"`	Multi-class cross-entropy loss
`model.fit(..., epochs=100)`	100 passes through training data; backpropagation each batch
`Backpropagation steps`	Forward → loss → backward (chain rule) → update

Running Procedure

1. Run: python program11_backpropagation_iris.py

2. Observe training progress and final accuracy

3. View loss and accuracy plots

Program 12: SGD vs SGD with Momentum

Compare standard SGD vs SGD with momentum on the same dataset.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD

X, y = make_classification(n_samples=1000, n_features=10, random_state=42)
y = y.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test)

def build_model():
    model = Sequential([Dense(16, activation='relu', input_shape=(10,)),
                        Dense(16, activation='relu'),
                        Dense(1, activation='sigmoid')])
    return model

# SGD without momentum
model_sgd = build_model()
model_sgd.compile(optimizer=SGD(learning_rate=0.01, momentum=0.0),
                  loss='binary_crossentropy', metrics=['accuracy'])
h_sgd = model_sgd.fit(X_train, y_train, epochs=30, batch_size=32,
                      validation_data=(X_test, y_test), verbose=0)

# SGD with momentum
model_mom = build_model()
model_mom.compile(optimizer=SGD(learning_rate=0.01, momentum=0.9),
                  loss='binary_crossentropy', metrics=['accuracy'])
h_mom = model_mom.fit(X_train, y_train, epochs=30, batch_size=32,
                      validation_data=(X_test, y_test), verbose=0)

# Compare
print(f"{'Optimizer':20} {'Train Acc':12} {'Test Acc':12} {'Epochs to 90%':16}")
print("-" * 60)
sgd_epochs = np.argmax(h_sgd.history['accuracy'] >= 0.9) + 1 if any(a >= 0.9 for a
in h_sgd.history['accuracy']) else 'Never'
mom_epochs = np.argmax(h_mom.history['accuracy'] >= 0.9) + 1 if any(a >= 0.9 for a
in h_mom.history['accuracy']) else 'Never'
print(f"{'SGD (momentum=0)':20} {h_sgd.history['accuracy'][-1]:12.4f}
{h_sgd.history['val_accuracy'][-1]:12.4f} {str(sgd_epochs):>16}")
print(f"{'SGD (momentum=0.9)':20} {h_mom.history['accuracy'][-1]:12.4f}
{h_mom.history['val_accuracy'][-1]:12.4f} {str(mom_epochs):>16}")
```

```
plt.figure(figsize=(12, 4))
plt.subplot(121)
plt.plot(h_sgd.history['loss'], 'b-', label='SGD Loss')
plt.plot(h_mom.history['loss'], 'r-', label='SGD+Momentum Loss')
plt.xlabel('Epoch'); plt.ylabel('Loss'); plt.legend(); plt.grid(True)
plt.title('Training Loss Comparison')

plt.subplot(122)
plt.plot(h_sgd.history['accuracy'], 'b-', label='SGD Acc')
plt.plot(h_mom.history['accuracy'], 'r-', label='SGD+Momentum Acc')
plt.xlabel('Epoch'); plt.ylabel('Accuracy'); plt.legend(); plt.grid(True)
plt.title('Training Accuracy Comparison')
plt.tight_layout(); plt.show()
```

Line-by-Line Explanation

`SGD(learning_rate=0.01, momentum=0.0)`	**Standard SGD: no momentum; slower convergence**
`SGD(learning_rate=0.01, momentum=0.9)`	SGD with momentum: adds 90% of previous update to current
`momentum=0.9`	Update = lr*gradient + momentum*previous_update; accelerates in consistent directions
`np.argmax(accuracy >= 0.9)`	Find first epoch where accuracy reaches 90%
`build_model()`	Helper to create identical model for fair comparison

Running Procedure

1. Run: python program12_sgd_comparison.py

2. Observe that momentum converges faster to higher accuracy

3. View comparison table and plots

Program 13: Model Complexity — Training vs Test Error

Analyze how different model complexities affect training and test errors.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

X, y = make_classification(n_samples=500, n_features=10, random_state=42)
y = y.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train); X_test = scaler.transform(X_test)

# Test different hidden layer sizes (model complexity)
hidden_sizes = [2, 4, 8, 16, 32, 64, 128, 256]
train_errors, test_errors = [], []

for size in hidden_sizes:
    model = Sequential([
        Dense(size, activation='relu', input_shape=(10,)),
        Dense(size, activation='relu'),
        Dense(1, activation='sigmoid')
    ])
    model.compile(optimizer=Adam(0.001), loss='binary_crossentropy',
metrics=['accuracy'])
    history = model.fit(X_train, y_train, epochs=30, batch_size=32,
                        validation_data=(X_test, y_test), verbose=0)
    train_acc = history.history['accuracy'][-1]
    test_acc = history.history['val_accuracy'][-1]
    train_errors.append(1 - train_acc)
    test_errors.append(1 - test_acc)
    print(f"Hidden size={size:3d}  Train err={1-train_acc:.4f}  Test err={1-
test_acc:.4f}  Gap={test_acc-train_acc:+.4f}")

# Plot
plt.figure(figsize=(10, 6))
plt.plot(hidden_sizes, train_errors, 'bo-', label='Training Error', linewidth=2)
plt.plot(hidden_sizes, test_errors, 'rs--', label='Test Error', linewidth=2)
plt.xlabel('Hidden Layer Size (Complexity)')
plt.ylabel('Error Rate')
plt.title('Model Complexity: Underfitting vs Overfitting')
plt.legend(); plt.grid(True, alpha=0.3)
plt.xscale('log')
plt.axvline(x=hidden_sizes[np.argmin(test_errors)], color='g', linestyle=':',
alpha=0.7)
```

```
plt.text(hidden_sizes[np.argmin(test_errors)], 0.02, ' Sweet Spot', fontsize=10,
color='green')
plt.show()
```

Line-by-Line Explanation

`hidden_sizes = [2, 4, 8, 16, 32, 64, 128, 256]`	**Increasing model complexity from 2 to 256 hidden neurons**
`train_errors = 1 - train_acc`	Error rate = 1 minus accuracy
`plt.xscale("log")`	Log scale on x-axis to spread out small values
`Underfitting (small size)`	High training error: model too simple to learn patterns
`Overfitting (large size)`	Low training error but high test error: model memorizes noise
`Sweet spot`	Hidden size with lowest test error — optimal complexity

Running Procedure

1. Run: python program13_model_complexity.py

2. Observe the underfitting → sweet spot → overfitting pattern

3. View plot showing training vs test error trade-off

Program 14: Simple CNN for Image Classification

Build a CNN for image classification on CIFAR-10 or MNIST.

Complete Program

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.utils import to_categorical

# Load CIFAR-10 (smaller subset for speed)
(X_train, y_train), (X_test, y_test) = cifar10.load_data()
X_train, X_test = X_train[:10000]/255.0, X_test[:2000]/255.0
y_train, y_test = y_train[:10000], y_test[:2000]
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

# Build CNN
model = Sequential([
    Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    MaxPooling2D((2,2)),
    Conv2D(64, (3,3), activation='relu'),
    MaxPooling2D((2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
model.summary()

# Train
history = model.fit(X_train, y_train_cat, epochs=5, batch_size=64,
                    validation_data=(X_test, y_test_cat), verbose=1)

# Evaluate
test_loss, test_acc = model.evaluate(X_test, y_test_cat, verbose=0)
print(f"\nTest Accuracy: {test_acc:.4f}")

# Predictions
preds = model.predict(X_test[:10])
pred_classes = np.argmax(preds, axis=1)

plt.figure(figsize=(12, 5))
for i in range(10):
    plt.subplot(2, 5, i+1)
```

```
        plt.imshow(X_test[i])
        color = 'green' if pred_classes[i] == y_test[i,0] else 'red'
        plt.title(f"True: {class_names[y_test[i,0]]}\nPred:
    {class_names[pred_classes[i]]}", color=color, fontsize=9)
        plt.axis('off')
    plt.tight_layout(); plt.show()
```

Line-by-Line Explanation

`cifar10.load_data()`	**CIFAR-10: 60k 32x32 color images, 10 classes**
`X_train[:10000]/255.0`	Use 10k subset for speed; normalize pixels to [0,1]
`Conv2D(32, (3,3), activation="relu")`	32 filters of size 3x3; detects features like edges, textures
`MaxPooling2D((2,2))`	Downsample 2x2 → keeps max, reduces spatial size by 75%
`Flatten()`	Convert 2D feature maps to 1D vector for dense layers
`Dropout(0.5)`	Randomly drop 50% of neurons during training (prevents overfitting)
`Dense(10, activation="softmax")`	10 output classes with probability distribution

Running Procedure

1. Run: python program14_cnn.py (may take 2-5 minutes)

2. Observe model summary and training progress

3. View sample predictions (green=correct, red=wrong)

Program 15: Prompt for Generating MLP Notes

Use this prompt with any LLM (ChatGPT, Claude, etc.) to generate detailed notes on MLP.

Prompt to use with LLM

```
Write comprehensive textbook-style notes on Multilayer Perceptron (MLP) covering:

1. Architecture: Input layer, hidden layers, output layer. How information flows forward.
2. Activation Functions: Why non-linear activation is necessary. Compare ReLU, Sigmoid, Tanh.
3. Forward Propagation: Mathematical formulation. Weighted sum → activation → output.
4. Loss Functions: MSE for regression, cross-entropy for classification.
5. Backpropagation: Chain rule of calculus. Computing gradients for each weight.
6. Weight Updates: Gradient Descent. Learning rate. Update rule: w = w - lr * dL/dw.
7. Vanishing/Exploding Gradients: Causes and solutions (ReLU, Batch Normalization, proper initialization).
8. Regularization: L1/L2, Dropout, Early Stopping.
9. Hyperparameters: Number of layers, neurons per layer, learning rate, batch size, epochs.
10. Practical Considerations: Data scaling, train/test split, monitoring overfitting.
11. Universal Approximation Theorem: MLP with one hidden layer can approximate any continuous function.

Include mathematical formulas, diagrams described in text, and a comparison table of activation functions. Target audience: undergraduate CS students.
```

Explanation

This prompt is designed for an LLM (ChatGPT, Claude, Gemini)	It requests comprehensive MLP notes in textbook style
11 key topics are specified	Ensures complete coverage of MLP theory and practice
The prompt asks for formulas, diagrams, and tables	Produces well-structured educational content
Target: undergraduate CS students	Appropriate depth and complexity level

Running Procedure

1. Copy the prompt above

2. Paste into ChatGPT, Claude, or any LLM

3. Review and save the generated notes

Program 16: Prompt for Summarizing a Research Paper

Use this prompt to summarize any ML/DL research paper.

Prompt to use with LLM

```
Please summarize the following research paper in a structured format:

Paper Title: [Insert paper title]
Authors: [Insert authors]

Please provide:

1. **Problem Statement**: What problem does this paper address? Why is it
important?
2. **Related Work**: How does this work build on or differ from previous
approaches?
3. **Proposed Method**: Detailed explanation of the approach/model/algorithm.
   - Architecture (if neural network): layers, activations, loss function
   - Training details: dataset, optimizer, hyperparameters
   - Key innovations
4. **Experiments**:
   - Datasets used and their sizes
   - Evaluation metrics
   - Baseline comparisons
   - Main results (with numbers)
5. **Ablation Studies**: What components were tested independently?
6. **Limitations**: What does the paper acknowledge as limitations?
7. **Key Takeaways**: 3-5 bullet points summarizing the most important
contributions.
8. **My Assessment**:
   - Strengths of the paper
   - Weaknesses or questions not addressed
   - Potential follow-up work

Format the summary in clear sections with bullet points for readability.
```

Explanation

Structured prompt with 8 sections	Ensures comprehensive paper summary
Sections cover problem, method, experiments, limitations	Critical analysis not just description
Includes "My Assessment" section	Encourages critical thinking about the paper
Adaptable to any paper	Fill in title and authors, paste the abstract/full text

Running Procedure

1. Copy the prompt above
2. Insert the paper title and authors
3. Paste the paper abstract or full text after the prompt
4. Send to an LLM to get a structured summary

General Setup Instructions

Install Required Libraries

One-time setup

```
pip install numpy pandas matplotlib scikit-learn tensorflow keras seaborn
```

Dataset Downloads

Program	Dataset	Source
1	diabetes.csv	Kaggle: Pima Indians Diabetes
2	Auto MPG	UCI (loaded via URL)
3	Breast Cancer	sklearn.datasets (built-in)
4	Glass	UCI (loaded via URL)
5	Bank Marketing	UCI (download manually)
6-13	Synthetic / built-in	Generated or sklearn datasets
14	CIFAR-10	keras.datasets (built-in)

Running Programs

- All programs are standalone .py files
- Run: python program_name.py
- Close plot windows to continue script execution
- Programs 10-14 may take 1-5 minutes to train

Troubleshooting

Issue	Solution
ModuleNotFoundError	pip install missing_package
URL dataset not loading	Download manually and update path in code
TensorFlow/Keras errors	pip install tensorflow
Slow training	Reduce epochs or use smaller data subset
CUDA/GPU errors	Add: import os; os.environ["CUDA_VISIBLE_DEVICES"]="-1"